

Department of Industrial Engineering

System Analysis Final Project

Object-Oriented System Analysis & Design

Report 1 - The “Story Scripting Editor”

Professor: Loke Kar Seng

Group Members:

F11301104 Gabriela Caceres

F11301101 Juan Azcona

F11301111 Filippo Maldonado

F11301114 Sofia Sotelo

Table of Contents

- 1. Introduction & Project Scope**
- 2. Requirements Analysis**
- 3. Use Case Modeling**
- 4. Interaction & Behavioral Modeling**
- 5. Class Diagram**
- 6. Dynamic Object Behavior**
- 7. Design Patterns & Architecture**

1. Introduction & Project Scope

1.1 Project Background

The Story Scripting Editor is a specialized desktop application designed to support writers, game designers, and screenwriters in the creation, organization, and management of structured narrative projects. The project was commissioned as part of the Object-Oriented System Analysis & Design course, with the course instructor acting as the client. Requirements were formally elicited through two live interview sessions conducted with the instructor, following structured preparation by each team member.

The primary inspiration for this system is the concept of an Obsidian-like note editor adapted specifically for narrative writing, one that allows authors to work non-linearly, managing individual story assets (scenes, characters, locations, and background context) as discrete, interconnected cards rather than as a single continuous document.

1.2 Project Objectives

The Story Scripting Editor aims to achieve the following objectives:

- Provide a card-based workspace where writers can create and manage scenes, characters, and world-building assets independently of one another.
- Enable writers to define and rearrange a narrative sequence by ordering scene cards without being locked into a linear writing flow.
- Allow tagging of scene cards by emotional value and action type to support visualization of the story's emotional arc.
- Support the use of placeholder IDs for characters and locations within scene text, which are automatically replaced with real names upon story compilation.
- Provide a validation mechanism that reports incomplete or structurally inconsistent story elements before export.

- Compile the ordered sequence of scenes into an exportable, human-readable story in formats such as plain text, Markdown, PDF, or JSON.
- Offer a dashboard providing at-a-glance metrics on the project's assets, including scene count, character count, word count, and validation status.

1.3 System Overview

At its core, the Story Scripting Editor is a card-based narrative management tool that allows writers to manage story assets independently and compile scenes into a defined linear sequence. A story project is composed of four primary types of assets, each represented as an editable card:

- **Scene Cards:** The atomic narrative units. Each scene card contains the written content of one scene, a title, a short description, an emotion tag, and an action type tag. Scenes are connected in a defined sequence to form the story flow.
- **Character Cards:** Global profiles attached to the project. Each character has fixed attributes (name, gender, race) and writer-defined attributes (personality, backstory, emotional background, and trust level).
- **Location Cards:** Named places referenced within scenes. Each location has a unique ID that can be embedded in scene text and replaced with its real name at compile time.
- **Background Cards:** Global world-building reference documents divided into thematic sections such as Geography, Technology, Historical Era, and Environment.

The system operates entirely offline as a single-user desktop application with local file storage. It does not require user authentication. Stories are organized as independent projects, and multiple projects can coexist within the editor simultaneously.

1.4 High-Level Scope

The Story Scripting Editor covers the full lifecycle of a narrative project, from asset creation to story export. Writers can create, edit, and delete scene, character, location, and background cards, tag scenes by emotional value and action type, and arrange them into a narrative sequence that can be compiled and exported in plain text, Markdown, PDF, or JSON formats. The system also includes story validation, lifecycle state management, a project dashboard, and search and filter capabilities.

The system operates as a single-user, offline desktop application with local file storage. Features such as multi-user collaboration, cloud storage, parallel branching within a single story, full version history, and a reader-facing interactive playback engine fall outside the scope of this project.

2. Requirements Analysis

2.1 Client Interview Summary

Requirements for the Story Scripting Editor were gathered through two formal elicitation sessions conducted with the course professor, who acted as the client. Each team member was assigned a specific domain of inquiry and prepared a set of targeted questions in advance. The first session covered system context, user roles, character management, core functionality, and typical usage flows. The second session refined and expanded on scene structure, background cards, validation behavior, versioning, and dashboard content.

The following key decisions emerged from the interviews: the system is a single-user, offline desktop application requiring no login or authentication. Stories are structured as independent projects composed of four card types, scenes, characters, locations, and background sections. Narrative flow is strictly linear within each Story. Alternative branches are represented as separate Story objects within the same Project, rather than as parallel paths inside one Story. Character and location names may be deferred during writing using placeholder IDs, which are automatically resolved to real names at compile time. The professor explicitly emphasized the importance of a validation report that identifies incomplete attributes, unconnected scenes, and unresolved IDs without blocking the writer from continuing work.

2.2 System Boundary Definition

The Story Scripting Editor operates as a closed, local desktop system. Its boundary encompasses everything involved in creating, organizing, validating, and exporting a narrative project on a single machine. The sole actor interacting with the system is the Writer, a non-technical user with no coding knowledge.

- **Within the system boundary:** project and card management, scene sequencing, emotional arc tagging, ID-based referencing, story validation, lifecycle state

transitions, compilation, export, auto-save, dashboard metrics, and search and filter functionality.

- **Outside the system boundary:** user authentication, multi-user collaboration, cloud or network storage, parallel branching within a single story, automated AI-driven narrative analysis, full version control with history diffing, multi-language content support, and any reader-facing interactive playback engine.

2.3 User Stories

The following user stories were derived from both client interview sessions. Each story follows the standard format: As a [role], I want to [action], so that [benefit]. Stories are grouped by functional area.

2.3.1 Project Management

ID	As a writer, I want to:	So that:	Priority
US - 01	create a new story project with a title	I can organize each narrative as an independent workspace	High
US - 02	open and switch between multiple story projects	I can work on several narratives in parallel without losing context	High
US - 03	have my work auto-saved continuously	I do not lose progress if the application crashes unexpectedly	High

US - 04	manually transition my story through its lifecycle states (Draft, Validated, Published)	I can track and communicate the current maturity of my narrative	Medium
---------	---	--	--------

2.3.2 Scene Management

ID	As a writer, I want to:	So that:	Priority
US - 05	create a scene card with a title, short description, and content text box	I can write and identify each scene independently	High
US - 06	edit and delete existing scene cards	I can refine or remove scenes as my story evolves	High
US - 07	tag each scene with an emotion type and an action type	I can classify scenes and later visualize the emotional arc of my story	High
US - 08	arrange scenes into a narrative sequence by drag-and-drop or by typing scene IDs	I can define the order in which scenes will appear in the compiled story	High
US - 09	embed Character IDs and Location IDs as placeholders inside scene text	I can write scenes before finalizing names, and have them resolved automatically at compile time	High

US - 10	leave scene fields blank and fill them in later	I am not forced to complete a card before saving and moving on	Medium
US - 11	preview the compiled story from the reader's perspective	I can experience the narrative as a reader would before exporting	Low

2.3.3 Character Management

ID	As a writer, I want to:	So that:	Priority
US - 12	create a character card with fixed attributes (name, gender, race) and custom attributes (personality, backstory, emotional background, trust level)	I have a centralized profile for each character in my story	Low
US - 13	assign a unique Character ID to each character	I can reference them as placeholders in scene text before finalizing their names	High
US - 14	set a character as not visible so they are hidden from the	I can keep inactive or unused characters out of the active	Low

	character list view across the project	workspace without deleting them	
US - 15	be warned if I try to delete a character that is still referenced in a scene	I do not accidentally create broken references in my narrative	Low
US - 16	be warned if I reference a Character ID in a scene that has not been created yet	I can catch missing character definitions early	Medium

2.3.4 Location & Background Management

ID	As a writer, I want to:	So that:	Priority
US - 17	create location cards with a unique ID and descriptive attributes	I can reference places as placeholders in scene text and resolve them at compile time	High
US - 18	create background cards divided into multiple thematic sections (Geography, Technology, Historical Era, Environment)	I have a structured world-building reference document for the entire project	Medium

US - 19	use Character IDs and Location IDs as references inside background card text	the background document stays consistent with the rest of the project	Medium
---------	--	---	--------

2.3.5 Validation & Compilation

ID	As a writer, I want to:	So that:	Priority
US - 20	run a validation check that reports blank fields, isolated scenes, and unresolved IDs without blocking my work	I can review and fix issues at my own pace without being forced to stop	High
US - 21	compile my ordered scenes into a single story with all IDs automatically replaced by their real names	I get a clean, readable version of my narrative ready for export	High
US - 22	be warned if a Character ID or Location ID used in a scene has no real name assigned yet at	I know which IDs still need to be resolved before the story is complete	High

	compile time		
US - 23	export the compiled story in plain text, Markdown, PDF, or JSON format	I can share or use the story outside of the editor	High

2.3.6 Dashboard & Search

ID	As a writer, I want to:	So that:	Priority
US - 24	view a dashboard showing total scene count, character count, word count, a character-scene map, emotion tag distribution, action tag distribution, and a list of incomplete scenes	I have a complete at-a-glance summary of my project's current state and can identify issues without running a full validation	Medium
US - 25	search for scenes and characters by name and ID	I can quickly locate a specific asset within a large project	Medium
US - 26	filter scenes and characters by multiple attributes	I can narrow down results without manually browsing all	Medium

	simultaneously (e.g., emotion tag and character involved)	cards	
--	---	-------	--

2.4 Non-Functional Requirements

The following non-functional requirements were established based on the client interviews and define the operational constraints of the system.

ID	Category	Requirement
NFR - 01	Platform	The system must run as a desktop application. No web or mobile version is required.
NFR - 02	Connectivity	The system must operate fully offline. No internet connection should be required for any feature.
NFR - 03	Storage	All data must be stored locally on the file system. No external database server is required.
NFR - 04	Usability	The system must be operable by a non-technical user with no coding knowledge. All interactions should be accessible through a graphical interface.
NFR - 05	Reliability	The system must auto-save

		work continuously so that data can be recovered after an unexpected crash.
NFR - 06	Scalability	The system should support up to 10,000 scenes and 1,000 characters per project without significant performance degradation.
NFR - 07	Technology stack	No specific programming language or framework is mandated by the client. The team may choose the technology stack freely.

3. Use Case Modeling

3.1 System Actor

The Story Scripting Editor has a single actor: the Writer. The Writer is a non-technical user with no coding knowledge who interacts with the system locally on a desktop machine. All use cases in the system are initiated by and visible to this actor. There is no secondary actor, system timer, or external service involved in any use case.

3.2 Use Case Overview

The sixteen use cases of the system are grouped into four functional areas as shown in the table below.

ID	Use Case	Description
UC - 01	Create project	Writer creates a new story project with a title.
UC - 02	Open project	Writer opens an existing project from local storage. Implied by the multi-project requirement confirmed in the interview
UC - 03	Manage story lifecycle	Writer manually transitions a story through DRAFT, VALIDATED, and PUBLISHED states.
UC - 04	Create Story Branch	This use case does not

		represent a full version-control system. It only creates a separate editable story branch with a parent reference and optional label.
UC - 05	Manage scene	Writer creates, edits, or deletes a scene card. Tagging (UC-09) is optionally triggered if the writer assigns emotion or action tags
UC - 06	Manage character	Writer creates, edits, or deletes a character card and its attributes.
UC - 07	Manage location	Writer creates, edits, or deletes a location card.
UC - 08	Manage background	Writer creates, edits, or removes background sections.
UC - 09	Tag scene	Writer assigns an emotion tag and action tag to a scene

		card.
UC - 10	Reorder scenes	Writer defines the narrative sequence of scenes via drag-and-drop or typed IDs.
UC - 11	Validate project	System checks for blank fields, unresolved IDs, and isolated scenes. Never blocks.
UC - 12	Compile story	System resolves all [CHAR:ID] and [LOC:ID] tokens and generates compiled text. Includes UC-11.
UC - 13	Preview story	Writer previews the compiled story in-app without exporting. Includes UC-12.
UC-14	Export Story	Writer exports the compiled story to local storage in TXT, Markdown, JSON, or PDF format.
UC-15	Search & Filter	Writer searches or filters

		scenes and characters by name, ID, emotion tag, or character reference.
UC-16	View Dashboard	Writer views project metrics such as scene count, character count, word count, tag distributions, and incomplete scenes.

3.3 Use Case Diagram

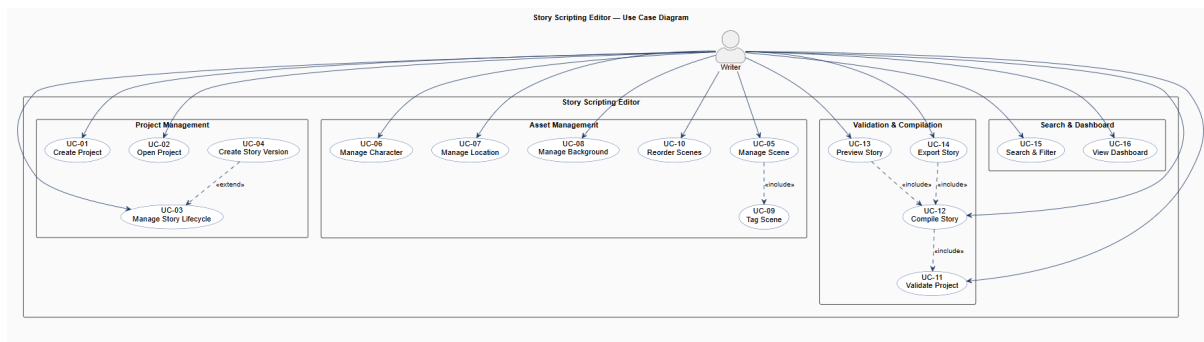


Figure 3.1 shows all sixteen use cases, their groupings, and their relationships (include and extend) within the system boundary.

3.4 Use Case Specifications

The following sections provide detailed textual specifications for the primary use cases of the system. Each specification follows the standard format: actor, preconditions, main success scenario, alternative flows, and postconditions.

UC-01: Create Project

Use Case ID	UC-01
Name	Create Project
Actor	Writer
Preconditions	The application is open. No preconditions on existing projects.
Main Success Scenario	1. Writer clicks 'New Project' in the project panel. 2. System prompts the Writer to enter a project title. 3. Writer enters a title and confirms. 4. System creates a new Project object with a unique ID, an empty story list, empty character and location lists, and an empty BackgroundStory. 5. System registers the Dashboard as an Observer on the new Project. 6. System auto-saves the project to local storage. 7. System navigates the Writer to the new project workspace.
Alternative Flows	A1 - Writer cancels: System discards the new project and returns to the previous state. A2 - Writer leaves title blank: System creates the project with a default title (e.g., 'Untitled Project').
Postconditions	A new Project exists in local storage and is visible in the project panel. Dashboard is active and showing zero metrics.

UC-03: Manage Story Lifecycle

Use Case ID	UC-03
Name	Manage Story Lifecycle
Actor	Writer
Preconditions	A Project is open and contains at least one Story.
Main Success Scenario	1. Writer selects a Story from the story list. 2. Writer clicks on the current lifecycle state indicator. 3. System displays the available target states (DRAFT, VALIDATED, PUBLISHED). 4. Writer selects a new state. 5. System calls Story.change_state(new_state) and updates the Story object. 6. System calls Project.notify_observers(). 7. Dashboard.update(project) recalculates and refreshes all metrics. 8. System auto-saves the project.
Alternative Flows	A1 - Writer selects the same state as current: System ignores the action. A2 - UC-04 extends this use case: Writer chooses to create a new version instead of overwriting the current state (see UC-04).
Postconditions	The Story's state attribute reflects the new value. Dashboard is updated.

UC-04: Create Story Branch

Use Case ID	UC-04
Name	Create Story Version
Actor	Writer
Extends	UC-03 (Manage Story Lifecycle)
Preconditions	A Project is open. A Story exists that the Writer wants to branch.
Main Success Scenario	1. Writer selects a Story and chooses 'Create New Version'. 2. System prompts the Writer to enter a version label. 3. Writer enters a label (e.g., 'Draft 2 alternate ending') and confirms. 4. System calls <code>Story.create_version(label)</code>, which creates a new Story object with <code>parent_version_id</code> set to the original Story's ID and state set to DRAFT. 5. System calls <code>Project.add_story(newStory)</code>. 6. System calls <code>Project.notify_observers()</code>. 7. <code>Dashboard.update(project)</code> refreshes metrics to include the new Story. 8. System auto-saves the project.
Alternative Flows	A1 - Writer cancels: No new Story is created. A2 - Writer leaves label blank: Version is created with a default label (e.g., 'Version 2').
Postconditions	A new Story object exists in the Project with <code>parent_version_id</code> pointing to the original Story. Both versions are independently editable.

UC-05: Manage Scene

Use Case ID	UC-05
Name	Manage Scene
Actor	Writer
Includes	UC-09 (Tag Scene)
Preconditions	A Project is open and a Story is selected.
Main Success Scenario	1. Writer clicks 'New Scene' in the scene panel. 2. System creates a Scene object with a unique auto-generated ID. 3. System opens the scene editor showing: ID (read-only), Title, Short Description, Content text box, Character Refs list, Location Refs list. 4. Writer fills in any desired fields (no field is mandatory see UC-11 for validation). 5. Writer adds Character IDs and Location IDs as placeholder references in the content or ref lists. 6. System triggers UC-09 (Tag Scene) so the Writer assigns emotion and action tags. 7. Writer saves the scene. 8. System calls <code>Story.add_scene(scene)</code>. 9. System calls <code>Project.notify_observers()</code> → <code>Dashboard.update(project)</code>. 10. System calls <code>Storage.auto_save(project)</code> to persist the updated project to local storage.
Alternative Flows	A1 - Writer edits an existing scene: Writer selects a scene from the list; steps 3–10 apply.

	A2 - Writer deletes a scene: System calls <code>Story.remove_scene(scene_id)</code> and notifies observers. A3 - Writer references a Character ID that does not exist: System does not block; UC-11 will flag this during validation.
Postconditions	The Scene exists in the selected Story's scene list. Dashboard metrics are updated.

UC-06: Manage Character

Use Case ID	UC-06
Name	Manage Character
Actor	Writer
Preconditions	A Project is open.
Main Success Scenario	1. Writer clicks 'New Character' in the character panel. 2. System creates a Character object with a unique auto-generated ID. 3. System opens the character editor showing: ID (read-only), Name, Visible flag, and a dynamic attributes section (personality, emotional background, trust level, backstory, and any writer-defined attributes). 4. Writer fills in the desired fields. 5. Writer saves the character. 6. System calls <code>Project.add_character(character)</code>. 7. System calls <code>Project.notify_observers()</code> → <code>Dashboard.update(project)</code>. 8. System auto-saves the project.

Alternative Flows	A1 - Writer edits an existing character: Writer selects a character; steps 3–8 apply. A2 - Writer attempts to delete a character that is referenced in one or more scenes: System calls <code>Character.is_referenced_in(scenes)</code>; if true, system blocks deletion and displays a warning. Character cannot be removed. A3 - Writer sets visible to false: Character no longer appears in the character list view across the project.
Postconditions	The Character exists globally in the Project. Dashboard character count is updated.

UC-11: Validate Project

Use Case ID	UC-11
Name	Validate Project
Actor	Writer
Included by	UC-12 (Compile Story)
Preconditions	A Project is open with at least one Story.
Main Success Scenario	- Writer clicks 'Validate' or system triggers this use case as part of UC-12. - System calls <code>Validator.validate(project)</code>. - Validator iterates all Stories → Scenes in the Project and checks: - Blank fields: scenes with empty title or content.

	b. Unresolved Character IDs: character_refs not matching any Character.id in the Project. c. Unresolved Location IDs: location_refs not matching any Location.id in the Project. d. Isolated scenes: scenes not included in any Story's ordered scene sequence. 4. Validator returns a ValidationReport with accumulated warnings and errors. 5. System displays the ValidationReport as a non-blocking popup. 6. Writer reviews the report and may continue editing without resolving issues.
Alternative Flows	A1 - No issues found: ValidationReport is empty; system displays a success message. A2 - Called from UC-12: If unresolved IDs exist, system warns the Writer before proceeding with compilation but does not block export.
Postconditions	A ValidationReport has been generated and presented. The Project state is unchanged regardless of validation outcome.

UC-12: Compile Story

Use Case ID	UC-12
Name	Compile Story
Actor	Writer

Includes	UC-11 (Validate Project)
Included by	UC-13 (Preview Story), UC-14 (Export Story)
Preconditions	A Project is open. A Story with at least one Scene in an ordered sequence exists.
Main Success Scenario	1. Writer initiates compilation (directly, or via UC-13/UC-14). 2. System executes UC-11 to validate the project and presents any warnings. 3. System calls StoryCompiler.compile(story, project). 4. StoryCompiler iterates the ordered Scene list of the selected Story. 5. For each Scene, StoryCompiler resolves [CHAR:ID] tokens by looking up Character.name in the Project. 6. StoryCompiler resolves [LOC:ID] tokens by looking up Location.name in the Project. 7. StoryCompiler concatenates all resolved scene contents into a single compiled string. 8. Compiled string is returned to the caller (UC-13 for preview, UC-14 for export).
Alternative Flows	A1 - A [CHAR:ID] or [LOC:ID] token has no matching name: System warns the Writer that unresolved tokens will appear as-is in the output but does not block compilation. A2 - Story has no scenes: System alerts the Writer that the story is empty and cancels compilation.

Postconditions	A compiled string representation of the story is available for preview or export.
-----------------------	---

UC-14: Export Story

Use Case ID	UC-14
Name	Export Story
Actor	Writer
Includes	UC-12 (Compile Story)
Preconditions	A Project is open with at least one Story containing ordered scenes.
Main Success Scenario	1. Writer clicks 'Export'. 2. System executes UC-12 to compile the story. 3. System presents a format selection dialog: .txt, .md, .json, or .pdf. 4. Writer selects a format and a destination path. 5. System calls StoryCompiler.export_to_file(content, path, format). 6. System writes the file to the chosen path on local storage. 7. System notifies the Writer that the export was successful.
Alternative Flows	A1 - Writer cancels format selection: Export is aborted; compiled content is discarded. A2 - File write fails (e.g., permission error): System displays an error message and allows the Writer to choose a different path.

Postconditions	A file in the chosen format exists at the specified path. The Project remains open and unchanged.
-----------------------	---

UC-15: Search & Filter

Use Case ID	UC-15
Name	Search & Filter
Actor	Writer
Preconditions	A Project is open with at least one Scene or Character.
Main Success Scenario	1. Writer enters one or more criteria in the search bar (e.g., emotion_tag = 'Catharsis', character_ref = 'C001'). 2. Writer selects whether to search Scenes or Characters. 3. System calls SearchService.search_scenes(criteria) or SearchService.search_characters(criteria). 4. SearchService navigates project.stories → story.scenes and filters results matching all provided criteria simultaneously. 5. System displays the filtered list of matching Scenes or Characters. 6. Writer clicks on a result to open the corresponding editor.
Alternative Flows	A1 No results match the criteria: System displays an empty result list with a 'No matches found' message. A2 Writer clears the search: System restores the full unfiltered list.
Postconditions	The UI shows the filtered result set. No data is modified.

UC-16: View Dashboard

Use Case ID	UC-16
Name	View Dashboard
Actor	Writer
Preconditions	A Project is open. Dashboard is registered as an Observer on the Project.
Main Success Scenario	1. Writer views the dashboard panel (always visible in the UI). 2. Dashboard displays current metrics: scene count, character count, word count, character-scene map, emotion tag distribution, action tag distribution, and list of incomplete scenes. 3. When the Writer modifies the project (adds/edits/deletes any asset), <code>Project.notify_observers()</code> is called automatically. 4. <code>Dashboard.update(project)</code> recalculates all metrics and refreshes the panel without any Writer action.
Alternative Flows	A1 Project is empty: Dashboard displays all metrics as zero.
Postconditions	Dashboard always reflects the current state of the Project at any given moment.

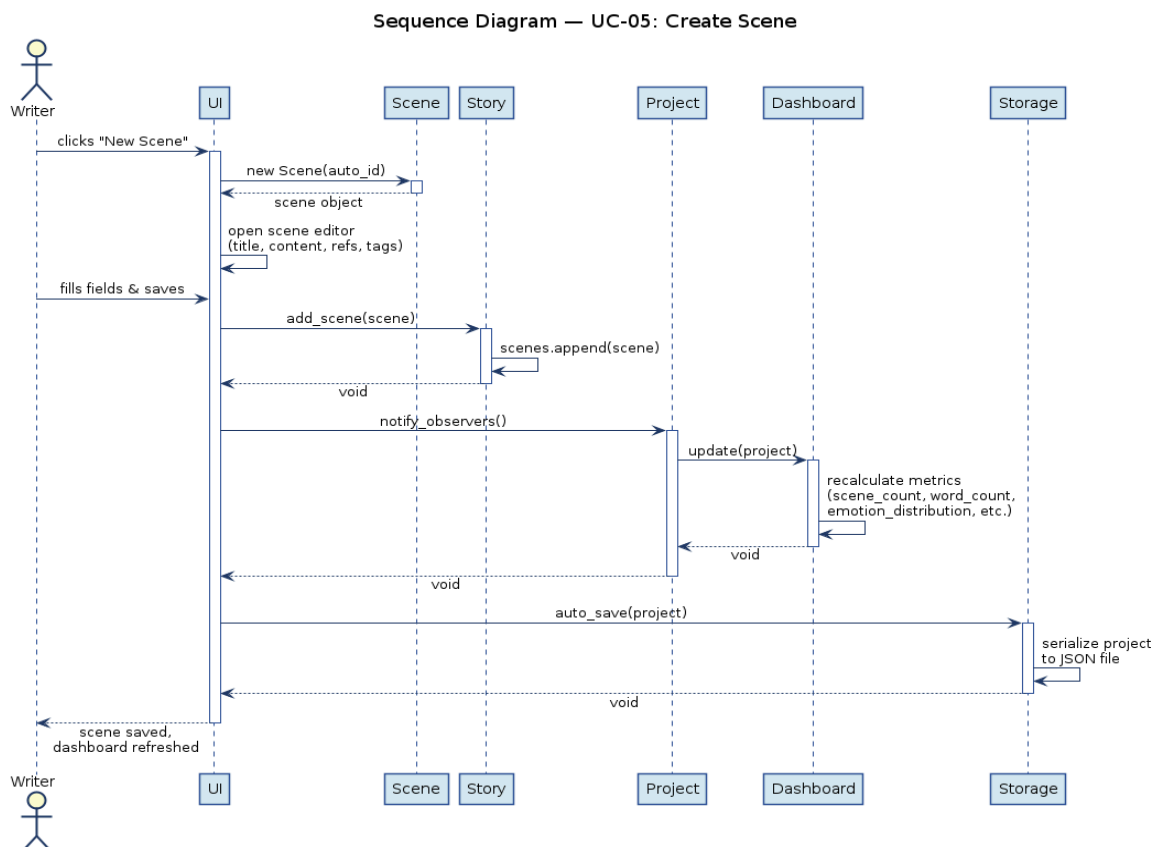
4. Interaction & Behavioral Modeling

4.1 Sequence Diagrams

Sequence diagrams capture the time-ordered exchange of messages between objects for critical use cases. Project-changing flows usually trigger `Project.notify_observers()`, which causes `Dashboard.update(project)` through the Observer Pattern. Auto-save is then handled separately through `Storage.auto_save(project)` as part of the persistence layer.

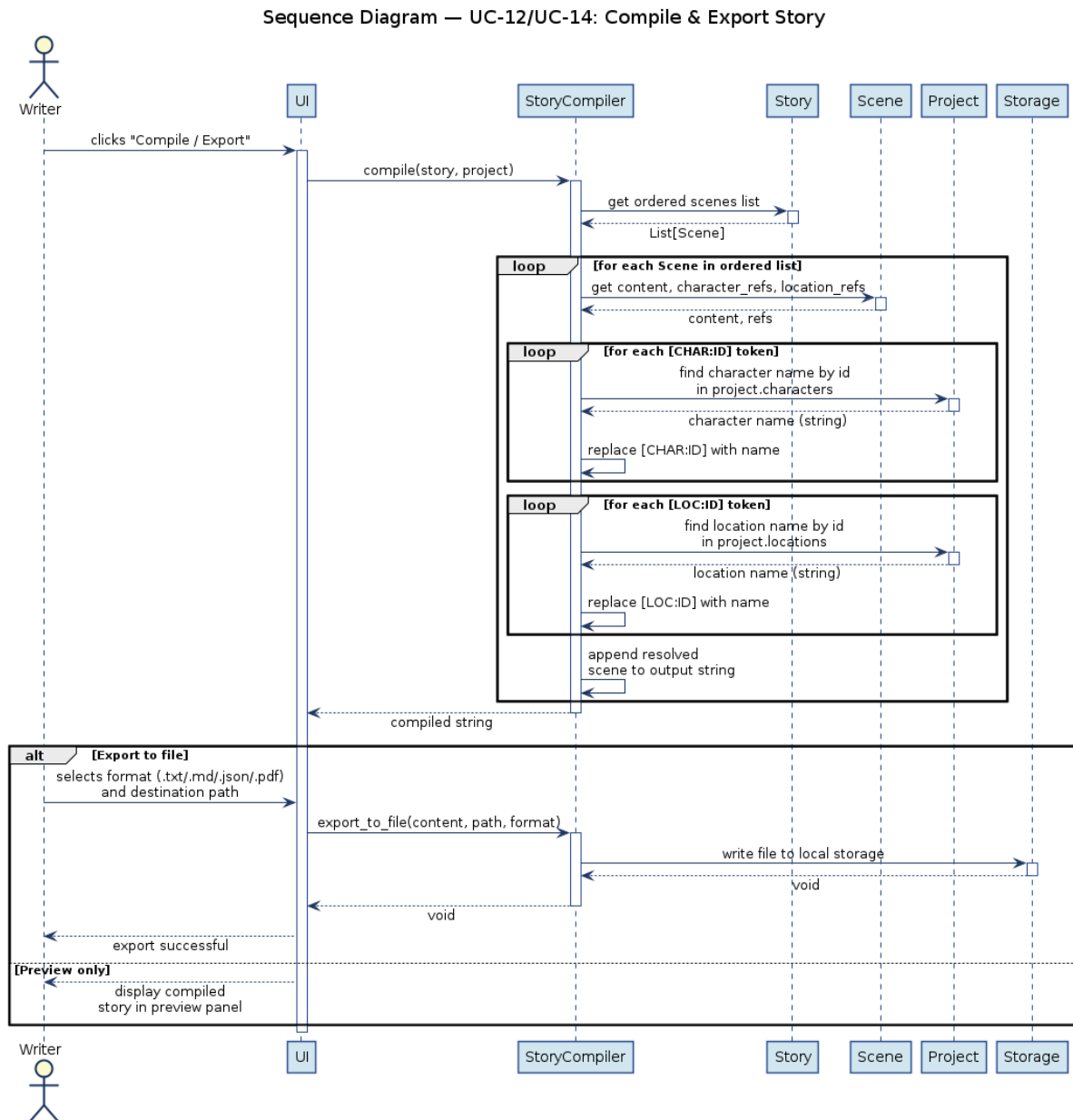
4.1.1 SD-01: Create Scene (UC-05)

This diagram shows the complete message flow when a Writer creates a new scene. It illustrates how the Scene object is instantiated, added to the Story, and how the Observer chain propagates the change to the Dashboard and Storage automatically.



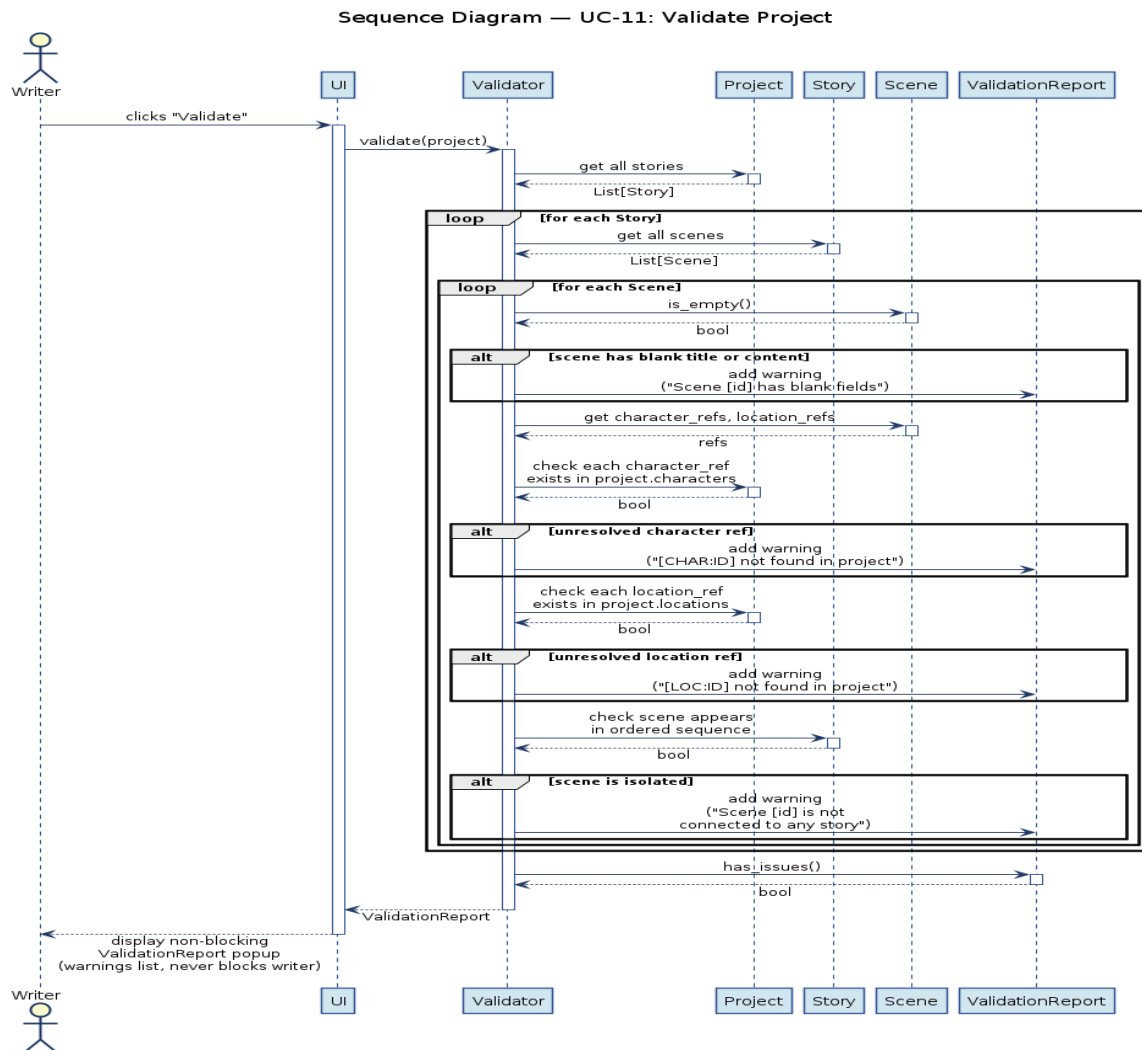
4.1.2 SD-02: Compile & Export Story (UC-12 / UC-14)

This diagram covers the compilation flow including the nested loops over scenes and ID tokens, the lookup of Character and Location names from the Project, and the branch between preview and file export.



4.1.3 SD-03: Validate Project (UC-11)

This diagram details how the Validator traverses all Stories and Scenes, checking for blank fields, unresolved Character and Location IDs, and isolated scenes, accumulating results in a ValidationReport that is presented to the Writer without blocking their workflow.



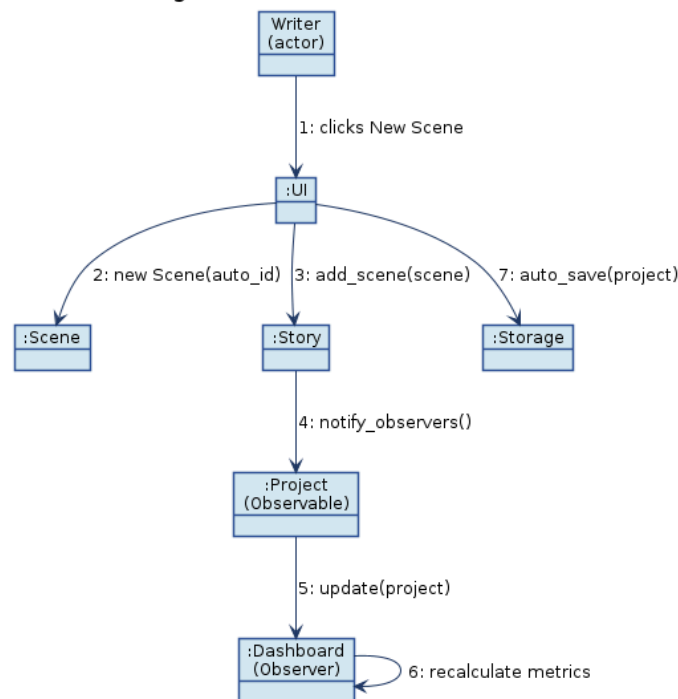
4.2 Communication Diagrams

Communication diagrams show the same interactions as sequence diagrams but focus on the structural links between objects and the numbered message sequence. They are particularly effective for visualizing the Observer pattern wiring in this system.

4.2.1 CD-01: Create Scene - Observer Pattern in Action

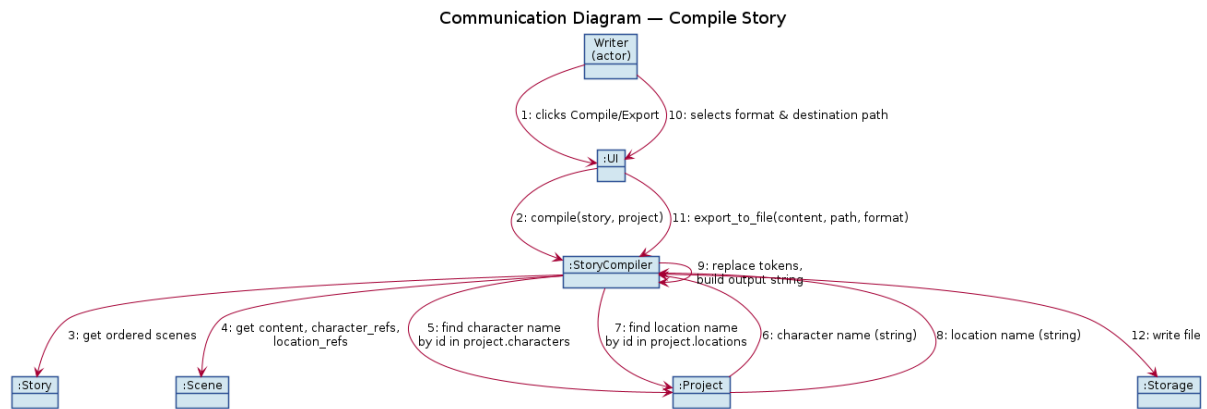
This diagram highlights how a single user action propagates through seven objects. The critical chain is messages 4→5→6: Story triggers `Project.notify_observers()`, which calls `Dashboard.update(project)`, demonstrating the loose coupling achieved by the Observer pattern.

Communication Diagram — Create Scene (Observer Pattern in action)



4.2.2 CD-02: Compile Story

This diagram shows how StoryCompiler navigates from Story to Scene to Project to Character and Location in sequence to resolve all placeholder tokens. The StoryCompiler acts as the central coordinator, never directly modifying the domain objects.

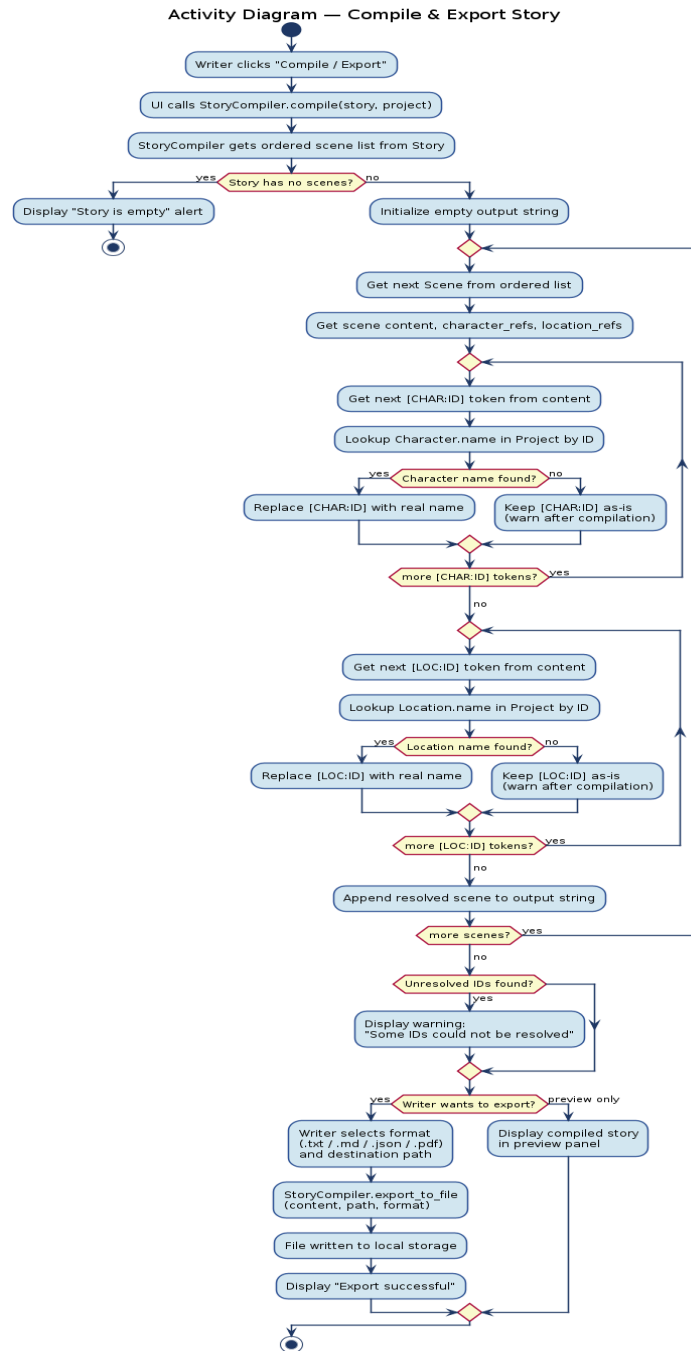


4.3 Activity Diagrams

Activity diagrams model the internal logic of the two most complex workflows in the system: compilation and validation. Both involve nested iteration and conditional branching that cannot be fully captured in a sequence diagram alone.

4.3.1 AD-01: Compile & Export Story

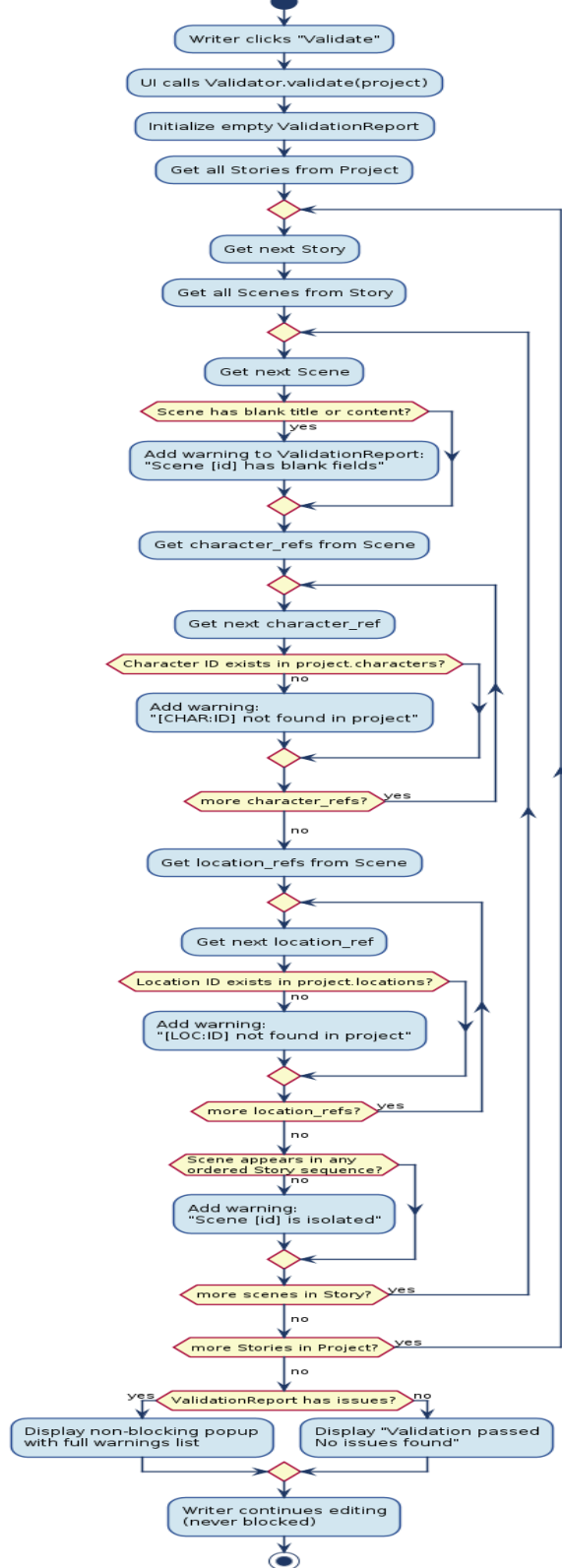
This activity diagram maps the full compilation workflow including the guard condition on empty stories, the nested loops over scenes and ID tokens, the conditional warning for unresolved IDs, and the final branch between preview and file export.



4.3.2 AD-02: Validate Project

This activity diagram maps the full validation workflow. It shows how the Validator traverses the three-level hierarchy of Project → Story → Scene, applies four distinct validation checks per scene, accumulates results non-destructively, and presents the final ValidationReport without ever blocking the Writer from continuing work.

Activity Diagram — Validate Project



5. Class Diagram

5.1 Overview

The class diagram for the Story Scripting Editor is organized into four packages: patterns, models, services, and ui. This separation reflects a clear layered architecture where domain entities, behavioral patterns, business logic, and interface concerns are kept independent of one another.

5.2 Package: patterns

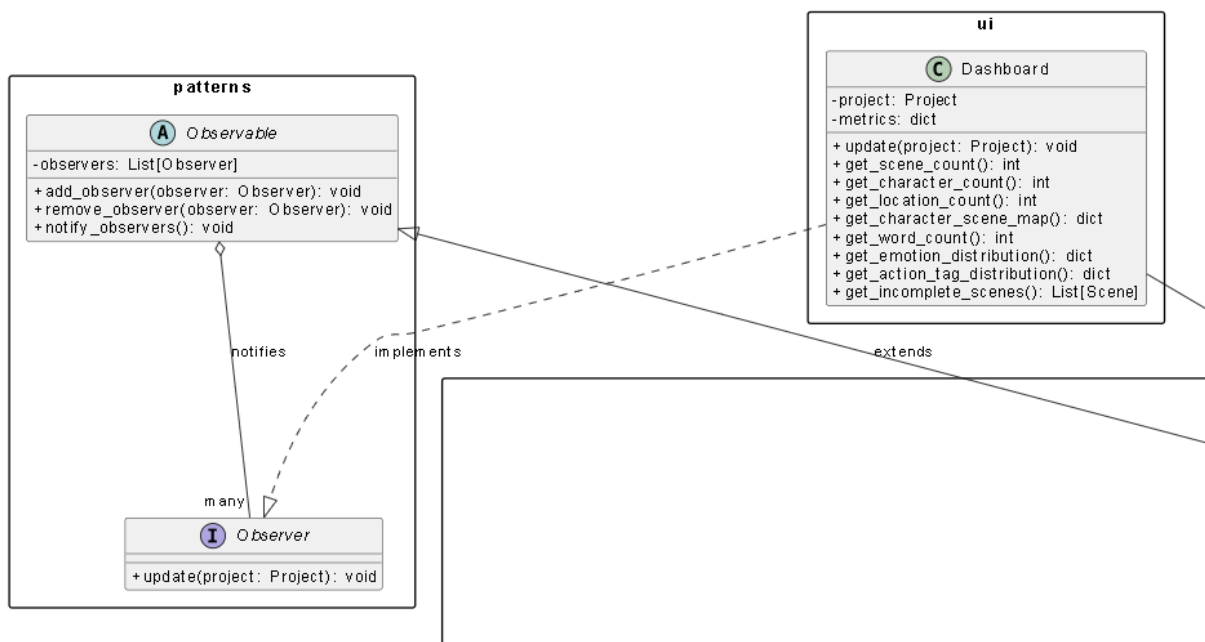


Figure 5.1 Observer Pattern Structure and Dashboard Integration

This package defines the structural foundation for the Observer design pattern used throughout the system.

Observable is an abstract class that maintains a list of registered observers and exposes three methods: `add_observer()`, `remove_observer()` and `notify_observers()`.

Any class that extends Observable automatically gains the ability to broadcast state changes to all registered listeners without knowing anything about them.

Observer is an interface with a single method: `update(project: Project)`. Any class that implements this interface agrees to receive and process project state changes whenever they are broadcast.

5.3 Package: models

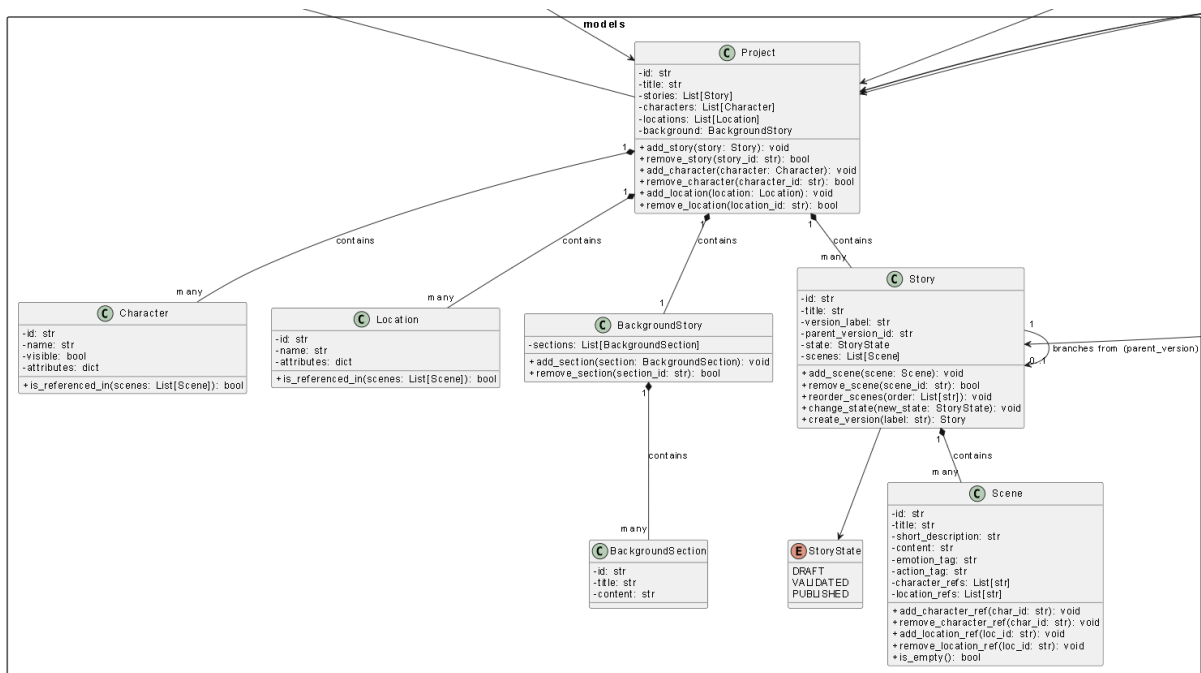


Figure 5.2 Domain Model Package: Project, Story, Scene, Character, Location, BackgroundStory, and BackgroundSection

This package contains all domain entities that represent the core data of a narrative project.

Project: is the central entity of the system. It extends `Observable`, which means every mutation it undergoes (adding or removing a story, character, or location) triggers a notification to all registered observers. A project holds four types of assets: a list of `Story` objects, a list of `Character` objects, a list of `Location` objects, and exactly one `BackgroundStory`. The composition relationships between `Project` and its contained entities mean that assets cannot exist outside of a project context.

Story: represents a single narrative thread within a project. Each story holds an ordered list of Scene objects and carries a lifecycle state (StoryState). The version_label and parent_version_id attributes enable story branching: a writer can call create_version(label) to derive a new Story from an existing one, producing a separate branch without modifying the original. This is how the system achieves branching while keeping the narrative within each individual story strictly linear.

StoryState: is an enumeration with three values DRAFT, VALIDATED, and PUBLISHED that models the maturity lifecycle of a story from initial writing to final release.

Scene: is the atomic unit of narrative content. It holds the written text (content), a title, a short_description, an emotion_tag, and an action_tag. Scenes reference characters and locations by ID (character_refs, location_refs) rather than by direct object reference. This design decision keeps scenes loosely coupled from character and location objects: a writer can embed a [CHAR:ID] token in scene text before the corresponding character has been defined, and the ID is only resolved to a real name at compile time by the StoryCompiler.

Character: represents a narrative actor within a project. Characters are global to the project (not bound to any single story). The visible boolean flag allows a writer to hide a character from the active workspace without deleting them, a requirement confirmed explicitly by the professor. The attributes dictionary accommodates writerdefined fields such as personality, emotional background, trust level, and backstory, making the model flexible without requiring a fixed schema. The is_referenced_in(scenes) method enables the deleteguard logic: a character cannot be removed if they are still referenced in any scene.

Location: follows the same structural pattern as Character: it is global to the project, carries an attributes dictionary, and exposes is_referenced_in(scenes) to prevent deletion of locations that are still referenced.

BackgroundStory: is a single worldbuilding document attached to each project. It is composed of one or more BackgroundSection objects, each with a title and a content field that may contain [CHAR:ID] and [LOC:ID] tokens the same token syntax used in scenes so that background text stays consistent with the rest of the project at compile time.

5.4 Package: services

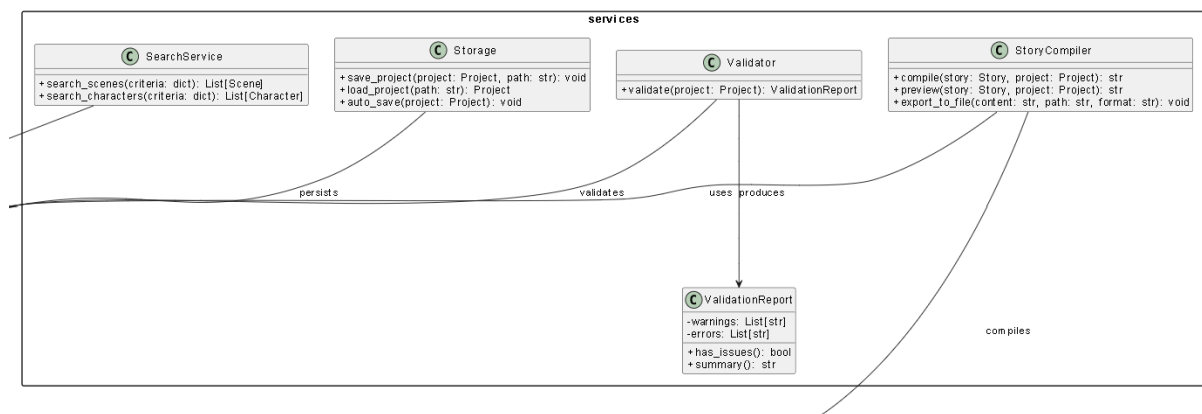


Figure 5.3 Services Package: Validator, StoryCompiler, SearchService, Storage, and ValidationReport

This package contains the business logic layer of the system. Services operate on model objects but do not own them; they are stateless processors that accept model instances as input and return results.

Validator: accepts a Project and returns a ValidationReport. It walks every story, scene, character, and location in the project and checks for blank fields, unresolved character or location IDs, and isolated scenes (scenes that have no sequence position). Importantly, the validator never blocks the writer: it reports issues as warnings and errors but does not prevent saving or continuing work.

ValidationReport: is the output object of the Validator. It holds two lists warnings and errors and exposes has_issues() and summary() as convenience methods for the UI to display results without processing the raw lists directly.

StoryCompiler: takes an ordered Story and the parent Project and replaces all [CHAR:ID] and [LOC:ID] tokens in scene content with the real names of the corresponding character and location objects. It exposes three methods: compile() for full compilation, preview() for an inapp read without writing to disk, and export_to_file() for writing the compiled output to a file in .txt, .md, .json, or .pdf format.

SearchService: provides multicriteria filtering over scenes and characters. search_scenes(criteria) accepts a dictionary of filter keys (such as emotion_tag, character_ref, or title) and returns only the scenes that match all supplied criteria simultaneously. search_characters(criteria) operates the same way over the character list. This design, confirmed by the professor, supports combining multiple filters in a single query.

Storage: handles all persistence. save_project() serializes a Project to JSON and writes it to a file path. load_project() reads and reconstructs a Project from a JSON file. auto_save() is triggered automatically after every project mutation to ensure crash recovery without any manual save action from the writer.

5.5 Package: ui

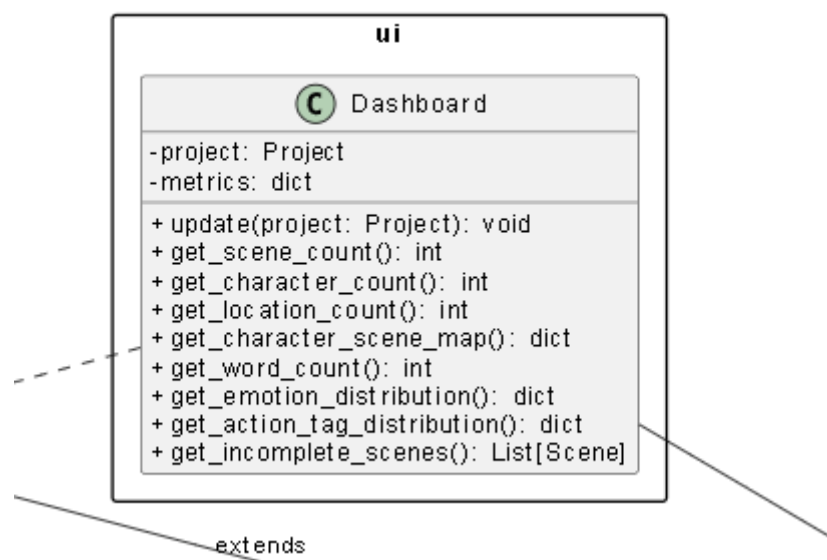


Figure 5.3 UI Package: Dashboard Class Implementing the Observer Interface

Dashboard: is the sole UI class represented in the class diagram. It implements the Observer interface, meaning it receives a Project instance every time the project changes and independently recalculates its metrics. It exposes methods for scene count, character count, word count, character-to-scene mapping, emotion tag distribution, action tag distribution, and incomplete scene listing. The Dashboard never asks the Project for an update; instead it waits passively for the Observer notification; this is what keeps the two classes loosely coupled.

5.6 Key Design Decisions

Three design decisions in the class diagram are worth noting explicitly.

First, scenes reference characters and locations by ID strings rather than by object reference. This allows scenes to be created before the referenced entities exist and supports the [CHAR:ID] / [LOC:ID] token system used by the compiler.

Second, branching is modeled at the Story level via `parent_version_id` rather than at the Scene level. This keeps scene sequencing strictly linear within each story, which was a hard constraint confirmed by the professor with no parallel branching within a story.

Third, Validator and StoryCompiler are separate service classes rather than methods on Project or Story. This keeps the domain model clean and makes each service independently testable without needing a full project object instantiated.

6. Dynamic Object Behavior

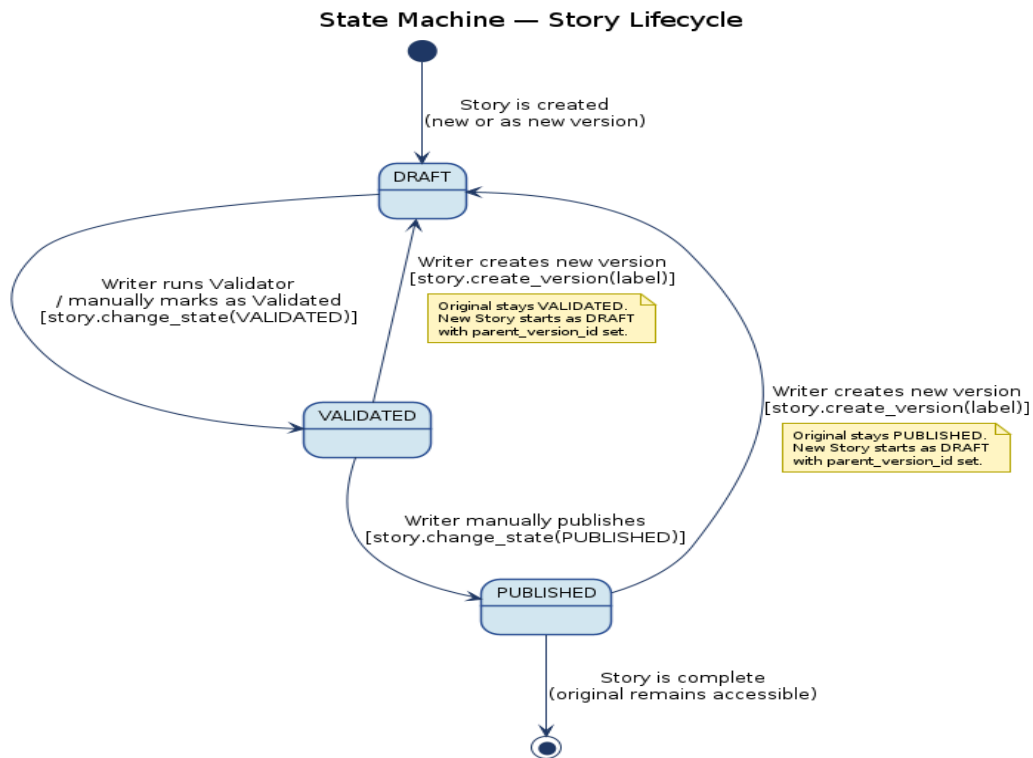
6.1 Overview

State machine diagrams model the lifecycle of objects whose behavior changes meaningfully over time in response to events or user actions. Unlike interaction diagrams, which show message flow between objects, state machines focus on a single object and the discrete states it can occupy, the transitions between those states, and the conditions or actions that trigger each transition.

Two objects in the Story Scripting Editor exhibit non-trivial lifecycle behavior that warrants explicit modeling. The Story object progresses through a formal approval workflow defined by the StoryState enum (DRAFT, VALIDATED, PUBLISHED), with transitions driven entirely by Writer decisions. The Scene object moves through completeness states based on how many of its optional fields the Writer has filled in, and its current state determines how the Validator reports it. Both lifecycles are documented in this chapter.

6.2 Story Lifecycle State Machine

The Story lifecycle is the primary behavioral model of the system and was explicitly requested as part of the design specification. A Story is always in one of three states defined by the StoryState enum: DRAFT, VALIDATED, or PUBLISHED. All transitions are manual, the Writer decides when to advance or branch a story; the system never changes the state automatically.

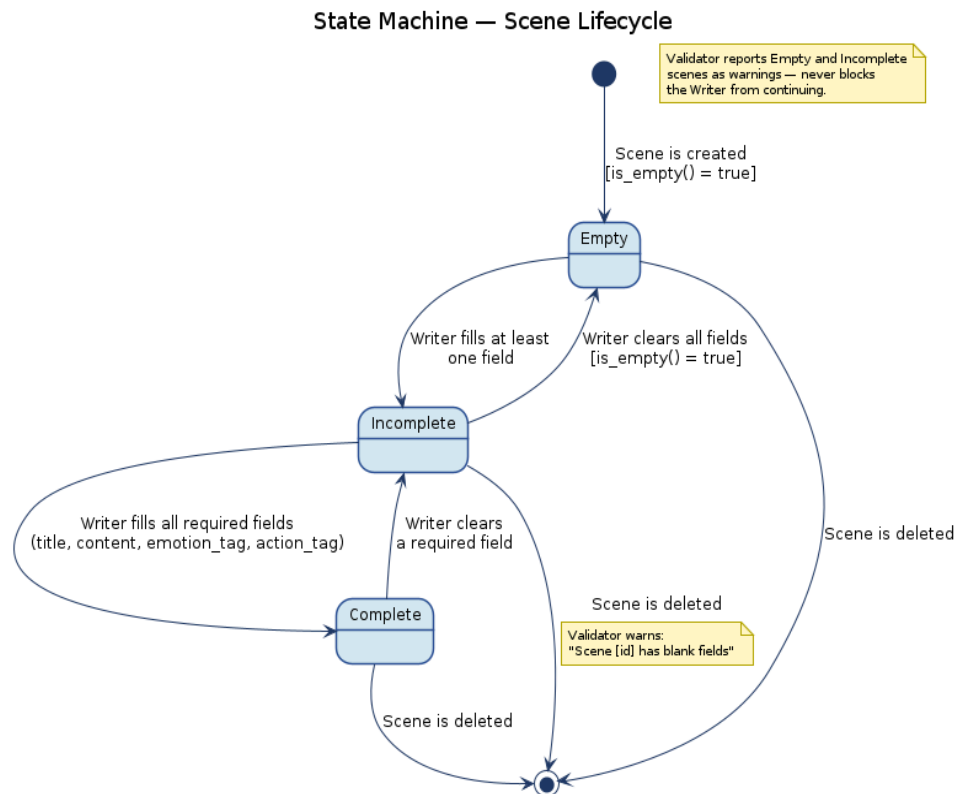


A Story enters existence in the DRAFT state, whether it is a brand-new story or a new version branched from an existing one. From DRAFT, the Writer can invoke `story.change_state(VALIDATED)` once the narrative is ready for review, this transition is also triggered when the Writer manually marks the story as Validated after running the Validator. From VALIDATED, the Writer can advance to PUBLISHED via `story.change_state(PUBLISHED)`, marking the story as the final approved version.

The system does not allow destructive backward transitions. If a Writer needs to revise a VALIDATED or PUBLISHED story, the correct action is `story.create_version(label)`, which creates a new Story object in DRAFT state with its `parent_version_id` pointing to the original. The original story remains unchanged in its current state, VALIDATED or PUBLISHED, and is never overwritten. This branching mechanism means that four narrative branches are represented as four distinct Story objects within the same Project, each with its own independent state.

6.3 Scene Lifecycle State Machine

The Scene lifecycle models the completeness of a scene's content as the Writer fills in its fields. Because no fields are mandatory at save time, a Scene can remain in any state indefinitely. The three states, Empty, Incomplete, and Complete are derived from the presence or absence of the four key fields: title, content, emotion_tag, and action_tag.



A Scene starts in the Empty state immediately after creation, when `is_empty()` returns true (both title and content are absent). As soon as the Writer fills in at least one field, the Scene transitions to Incomplete. Once all four required fields title, content, emotion_tag, and action_tag are present, the Scene reaches the Complete state. Transitions are fully reversible, clearing a required field moves a Complete scene back to Incomplete, and clearing all fields returns it to Empty.

The Validator interacts with this lifecycle but never controls it. When `Validator.validate(project)` runs, it inspects each Scene and adds a warning to the

ValidationReport for every scene found in the Empty or Incomplete state. These warnings are presented to the Writer in a non-blocking popup, the Writer can acknowledge them and continue editing without taking any action. A Scene can be deleted from any state, and an Incomplete scene deletion will also produce a Validator warning if validation is run afterward.

7. Design Patterns & Architecture

Observer Pattern applied to the Story Scripting Editor (Chapter 7)

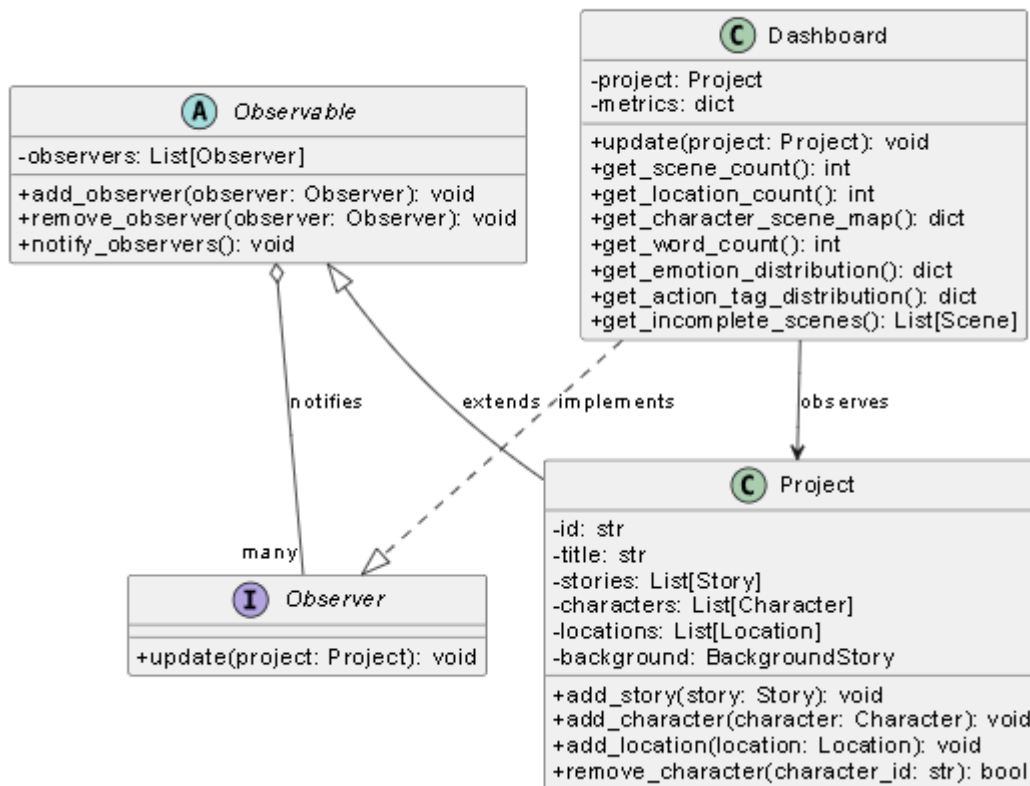


Figure 7.1 Observer Pattern Applied to the Story Scripting Editor

7.1 Architectural Overview

The Story Scripting Editor follows a layered object-oriented architecture composed of four main packages: patterns, models, services, and ui. The models package contains the core domain entities of the system, including Project, Story, Scene, Character, Location, BackgroundStory, and BackgroundSection. These classes represent the data that the writer creates and manages inside the editor.

The services package contains the main business logic of the system. Validator is responsible for checking incomplete fields, unresolved IDs, and isolated scenes. StoryCompiler compiles the ordered scenes into a readable output and replaces placeholder

IDs with real names. SearchService supports multi-criteria search and filtering, while Storage manages local persistence and auto-save.

The ui package contains interface-side components, represented in the class diagram by Dashboard. The dashboard provides project metrics such as scene count, character count, word count, tag distributions, and incomplete scene warnings. Finally, the patterns package contains the reusable Observer structure that supports automatic dashboard synchronization.

7.2 Selected Design Pattern: Observer Pattern

The primary design pattern used in the Story Scripting Editor is the Observer Pattern. This pattern was selected because the system requires the dashboard to stay synchronized with the current state of the project. Whenever the writer adds, edits, or deletes scenes, characters, locations, or background sections, the dashboard must automatically refresh its metrics.

In this design, Observable defines the generic structure for maintaining a list of observers and notifying them when a change occurs. Project acts as the concrete subject because it owns the main project data and broadcasts updates when its contents change. Observer defines the update(project) operation, and Dashboard acts as the concrete observer, receiving project updates and recalculating its displayed metrics.

This design keeps the system loosely coupled. Project does not need to know how the dashboard calculates or displays its information. It only needs to notify all registered observers that a change has happened.

In the prototype, the Observer Pattern is most visible in dashboard behavior: the writer modifies project data, the project broadcasts the change, and the dashboard updates without the project directly controlling the dashboard interface.

7.3 Why the Observer Pattern Fits the System?

The Observer Pattern is appropriate for this system because the dashboard must reflect project changes automatically and consistently. Without the pattern, the Project class would need to call dashboard-specific update logic directly every time a story asset changed. That approach would tightly couple the domain model to the interface layer and make the design harder to extend or maintain.

By using the Observer Pattern, the responsibilities remain clearly separated. Project is responsible only for maintaining project data and broadcasting changes. Dashboard is responsible only for calculating and displaying metrics. This separation improves modularity and supports future extension. For example, additional observers such as an activity log, validation status monitor, or save-status indicator could be introduced later without modifying the Project class.

The pattern also fits the behavior described in the interaction diagrams. In several use cases, project mutations trigger `Project.notify_observers()`, which causes `Dashboard.update(project)` to refresh the displayed metrics. Auto-save is handled as a separate persistence responsibility through `Storage.auto_save(project)`, while the Observer Pattern specifically explains the loose coupling between Project and Dashboard.

7.4 Pattern Role Mapping

Table 7.1 Observer Pattern Applied to the Story Scripting Editor

Pattern Role	Class in the System	Responsibility
Subject / Observable	Observable	Maintains the list of observers and provides <code>add_observer()</code> , <code>remove_observer()</code> , and <code>notify_observers()</code>

Concrete Subject	Project	Owens project data and broadcasts updates whenever stories, scenes, characters, locations, or background data are modified
Observer Interface	Observer	Declares the update(project) operation
Concrete Observer	Dashboard	Receives project updates and recalculates metrics such as scene count, character count, word count, tag distributions, and incomplete scenes

7.5 Alternative Patterns Considered

Three other design patterns were considered during the design process.

The Command Pattern was considered as an approach for encapsulating operations as objects, which naturally supports undo and redo behavior. It was rejected because the professor explicitly confirmed during the requirements interview that no revision history or undo system is required in this project. Introducing Command would add significant structural complexity for a feature that is out of scope.

The Strategy Pattern was considered for the compilation and validation logic, since it allows algorithms to be swapped at runtime through a common interface. It was rejected because the system has exactly one compilation algorithm and one validation algorithm with no interchangeable variants. The flexibility Strategy provides would go unused, making it unnecessary overhead for this design.

The State Pattern was also relevant because the system includes a story lifecycle. However, the lifecycle is simple enough to be represented by the StoryState enumeration with the values DRAFT, VALIDATED, and PUBLISHED, without requiring the additional complexity of full state objects and transition classes.

The Observer Pattern was selected because it directly addresses the most important architectural need of the system: keeping the Dashboard synchronized with the Project automatically while preserving loose coupling between the domain model and the interface layer.

The Observer Pattern plays a central role in the Story Scripting Editor architecture. It supports automatic dashboard updates, preserves separation of concerns, and allows the domain model to remain independent from the interface layer. Combined with the layered package structure of models, services, patterns, and ui, the pattern produces an architecture that is modular, understandable, and aligned with the object-oriented analysis and design goals of the project.